PropertyViewerWithSelection コンポーネント UI 仕様書

1. 概要

1.1 コンポーネントの目的

PropertyViewerWithSelectionは、PropertyViewerを拡張したコンポーネントで、チェックボックスによるプロパティ選択機能を追加したものです。選択されたプロパティはノード上に表示されます。

1.2 使用場面

- PetriNetViewerでPlaceノードをクリックした際のプロパティ選択・表示
- Drawer内でのプロパティ選択UI
- ノードに表示するプロパティのカスタマイズ

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

2.1.1 通常表示時

```
┌──────────────────────────────────────────────────────┐
│ [ノード名] プロパティ  [Material]        [×] 閉じる    │ ← ヘッダー（色付き）
├──────────────────────────────────────────────────────┤
│ [基本情報 2] [テンプレート 1] [データ]  ← タブ + 選択数  │
├──────────────────────────────────────────────────────┤
│ □  プロパティ              値                        │
│ ──────────────────────────────────────────    │
```

```
| ☑  ID                    place_001                    |
| ☑  名前                  サンプル材料                  |
| ☐  タイプ                Material                     |
| ☐  説明                  これはサンプル材料です        |
|                                                       |
└──────────────────────────────────────────────────────┘
```

2.1.2 プロパティなし

```
┌──────────────────────────────────────────────────────┐
| [ノード名] プロパティ  [Material]        [×] 閉じる   |
├──────────────────────────────────────────────────────┤
| [基本情報] [テンプレート] [データ]                     |
├──────────────────────────────────────────────────────┤
|                                                       |
|              プロパティがありません                   |
|                                                       |
└──────────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 ヘッダー

- **背景色**: ノードタイプによって変化（PropertyViewerと同じ）
  - **Material**: #F8C471（黄色）
  - **Condition**: #85C1E9（シアン）
  - **Result**: #D7BDE2（マゼンタ）
  - **Transition**: #CCCCCC（グレー）
- **タイトル**: `[ノード名] プロパティ`（Title level 5、白文字）
- **タグ**: ノードタイプ（白背景、ヘッダー色の文字）
- **閉じるボタン**: CloseOutlined アイコン、白文字

2.2.2 タブ（選択数表示付き）

- **基本情報**: ID、名前、タイプ、説明など + 選択数バッジ
- **テンプレート**: テンプレートプロパティ + 選択数バッジ
- **データ**: データプロパティ + 選択数バッジ
- **選択数バッジ**: 青色Tag、選択されたプロパティ数を表示

2.2.3 プロパティテーブル（チェックボックス付き）

- **カラム**:
  - チェックボックス（50px幅、中央揃え）
  - プロパティ名（35%幅、太字）
  - 値（自動幅）
- **ヘッダー**: 表示（チェックボックス列は空白）
- **サイズ**: small
- **ページネーション**: なし
- **スクロール**: 最大高さ400px

2.2.4 行のインタラクション

- **クリック**: 行全体がクリック可能、チェックボックスをトグル
- **選択状態**: 選択された行は背景色が青（#e6f7ff）
- **カーソル**: pointer

---

3. 機能仕様

3.1 プロパティ選択機能

3.1.1 チェックボックストグル

```
<Checkbox
  checked={selected.includes(record.key)}
  onChange={(e) => {
    e.stopPropagation();
    onPropertyToggle(structure, record.key);
  }}
/>
```

- **機能**: チェックボックスクリックでプロパティ選択をトグル
- **コールバック**: `onPropertyToggle(structure, property)` を呼び出し

- **stopPropagation**: チェックボックスクリックが行クリックと重複しないように

3.1.2 行クリック

```
onRow={(record) => ({
  onClick: () => onPropertyToggle(structure, record.key),
  style: {
    cursor: 'pointer',
    backgroundColor: selected.includes(record.key) ? '#e6f7ff' : 'transparent',
  },
})}
```

- **機能**: 行全体をクリックしてプロパティ選択をトグル
- **視覚フィードバック**: 選択状態に応じて背景色を変更

3.2 選択数の表示

3.2.1 処理フロー

```
const getSelectedCount = (structure: 'basic' | 'template' | 'data'): number => {
  return selectedProperties[structure]?.length || 0;
};

const tabItems = [
  {
    key: 'basic',
    label: (
      <Space>
        基本情報
        {getSelectedCount('basic') > 0 && (
          <Tag color="blue">{getSelectedCount('basic')}</Tag>
        )}
      </Space>
    ),
    // ...
  },
];
```

- **機能**: 各タブに選択されたプロパティ数を表示
- **表示条件**: 選択数が1以上の場合のみ表示

3.3 テーブルカラムの動的生成

3.3.1 処理フロー

```
const getColumns = (structure: 'basic' | 'template' | 'data'): ColumnsType<TableDataType> => {
  const selected = selectedProperties[structure] || [];

  return [
    {
      title: '',
      dataIndex: 'selected',
      key: 'selected',
      width: 50,
      align: 'center',
      render: (_: unknown, record: TableDataType) => (
        <Checkbox
          checked={selected.includes(record.key)}
          onChange={(e) => {
            e.stopPropagation();
            onPropertyToggle(structure, record.key);
          }}
        />
      ),
    },
    // ... その他のカラム
  ];
};
```

- **機能**: 構造（basic/template/data）に応じたカラム定義を生成
- **動的チェック**: 選択状態に基づいてチェックボックスの状態を設定

---

4. データ仕様

4.1 Props定義

```
interface PropertyViewerWithSelectionProps {
  properties: {
    id: string;
    name: string;
    type: 'Material' | 'Condition' | 'Result' | 'Transition';
  };
```

```
  structuredProperties: StructuredProperties;
  selectedProperties: SelectedProperties;
  onPropertyToggle: (
    structure: 'basic' | 'template' | 'data',
    property: string
  ) => void;
  onClose?: () => void;
}
```

Props説明:

Prop名	型	必須	説明
properties	Object	✅	ノードの基本情報（ID、名前、タイプ）
structuredProperties	StructuredProperties	✅	構造化されたプロパティ（Basic/Template/Data）
selectedProperties	SelectedProperties	✅	選択状態（親コンポーネントで管理）
onPropertyToggle	Function	✅	プロパティ選択トグル時のコールバック
onClose	Function	-	閉じるボタンクリック時のハンドラ

4.2 型定義

4.2.1 SelectedProperties

```
interface SelectedProperties {
  basic: string[];
  template: string[];
  data: string[];
}
```

説明:

フィールド	型	説明
basic	string[]	基本情報で選択されたプロパティキーの配列
template	string[]	テンプレートで選択されたプロパティキーの配列
data	string[]	データで選択されたプロパティキーの配列

4.2.2 PropertyObject（PropertyViewerと同じ）

```
interface PropertyObject {
  key?: string;
  value?: unknown;
  type?: string;
  description?: string;
  defaultValue?: string;
}
```

4.2.3 StructuredProperties（PropertyViewerと同じ）

```
interface StructuredProperties {
  basic: Record<string, PropertyObject>;
  template: Record<string, PropertyObject>;
  data: Record<string, PropertyObject>;
}
```

4.2.4 TableDataType

```
interface TableDataType {
  key: string;  // 内部キー（プロパティ名）
  propKey: string;  // 表示用プロパティ名
  value: unknown;  // 値
}
```

4.3 State定義

```
const [activeTab, setActiveTab] = useState('basic');
```

State説明:

State名	型	初期値	説明
activeTab	string	'basic'	アクティブなタブキー

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
-	（なし）	-	-	-	API呼び出しなし

備考:

- PropertyViewerWithSelectionはPropsとして渡されたデータを表示・操作するのみ
- 選択状態の管理とコールバックは親コンポーネント（PetriNetViewer）で実施

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
propertiesが空またはundefined	「プロパティ情報がありません」メッセージ表示
structuredPropertiesが空	Empty（「プロパティがありません」）表示

6.2 エラー処理コード

6.2.1 プロパティなし

```
if (!properties || !properties.id) {
  return (
    <div style={{ padding: 16 }}>
      <Text type="secondary">プロパティ情報がありません</Text>
    </div>
  );
}
```

6.2.2 テーブルデータなし

```
if (tableData.length === 0) {
  return (
    <Empty
      image={Empty.PRESENTED_IMAGE_SIMPLE}
      description="プロパティがありません"
    />
  );
}
```

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/dialogs/PropertyViewerWithSelection.tsx`

7.2 関連ファイル

- **ベースコンポーネント**: `frontend/src/components/viewer/dialogs/PropertyViewer.tsx`

7.3 依存コンポーネント

- なし（Ant Designコンポーネントのみ使用）

7.4 依存ライブラリ

- `antd` : Typography, Tabs, Table, Button, Space, Tag, theme, Checkbox, Empty
- `antd/es/table` : ColumnsType
- `@ant-design/icons` : CloseOutlined

---

8. 使用例

8.1 基本的な使用方法

```
import PropertyViewerWithSelection from '@/components/viewer/dialogs/PropertyViewerWithSelection';

const [selectedProperties, setSelectedProperties] = useState<SelectedProperties>({
  basic: ['id', 'name'],
  template: ['temperature'],
  data: []
});

const handlePropertyToggle = (
  structure: 'basic' | 'template' | 'data',
  property: string
) => {
  setSelectedProperties(prev => {
    const currentList = prev[structure] || [];
    const newList = currentList.includes(property)
      ? currentList.filter(p => p !== property)
      : [...currentList, property];

    return {
      ...prev,
      [structure]: newList
    };
  });
};

<PropertyViewerWithSelection
  properties={{
    id: 'place_001',
    name: 'サンプル材料',
    type: 'Material'
  }}
  structuredProperties={structuredProperties}
  selectedProperties={selectedProperties}
  onPropertyToggle={handlePropertyToggle}
  onClose={() => console.log('閉じる')}
/>
```

---

9. PropertyViewerとの違い

9.1 追加機能

機能	PropertyViewer	PropertyViewerWithSelection

プロパティ表示	✅	✅
チェックボックス選択	❌	✅
選択数バッジ	❌	✅
行クリック選択	❌	✅
選択状態の背景色変更	❌	✅

9.2 使い分け

- **PropertyViewer**: プロパティの閲覧のみ
- **PropertyViewerWithSelection**: プロパティの閲覧 + ノード表示用プロパティの選択

---

10. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline